Containerized RAG Log Investigator for an E-Commerce App with Local AI Inference

Rushikesh R. Shirsat
COMP 488 - Cloud Computing, DevOps, and AI,
Prof. Gregor von Laszewski, Ph.D.
Loyola University Chicago

Project summary

I will deploy an open-source PrestaShop e-commerce website with MySQL using Docker Compose on one Jetstream virtual machine. The store will generate application and database logs as I browse products and place test orders. Fluent Bit will automatically collect container logs and send them to a FastAPI log-investigator service. When I ask a question such as “Why did checkout fail?”, the service will retrieve relevant log events and ask a local AI model to explain them with timestamps and log evidence. The cloud VM will host the store and RAG services; Ollama and the language and embedding models will run on my departments DGX Spark over a private Tailscale connection. I will use synthetic test data and no real payment information.

Problem and motivation

E-commerce services depend on an application, database, and network working together. Their logs can show where a request failed, but connecting events across services can be difficult. This project demonstrates an end-to-end troubleshooting workflow while I learn Docker, cloud deployment, secure networking, CI/CD, vector search, and local AI inference.

Goals and success criteria

Run PrestaShop, MySQL, Fluent Bit, FastAPI, and Chroma as Docker services on a course-approved Jetstream VM. Automatically collect logs generated by store activity and container events; do not require manual log uploads. Ask plain-language questions and receive explanations that cite matching services, timestamps, severity, and log excerpts. Run Ollama and an embedding model on the DGX Spark, reachable from Jetstream only over Tailscale. Use GitHub Actions to test and build the project containers; deploy with a documented, repeatable process.
Demonstrate at least three controlled incidents and evaluate answers against a set of known outcomes. Evaluation target: prepare about 20 incident questions. For each response, check whether the explanation matches the known scenario, cites evidence that supports it, and admits when evidence is insufficient. Record response time and limitations; the target is for evaluation, not a promised accuracy level.

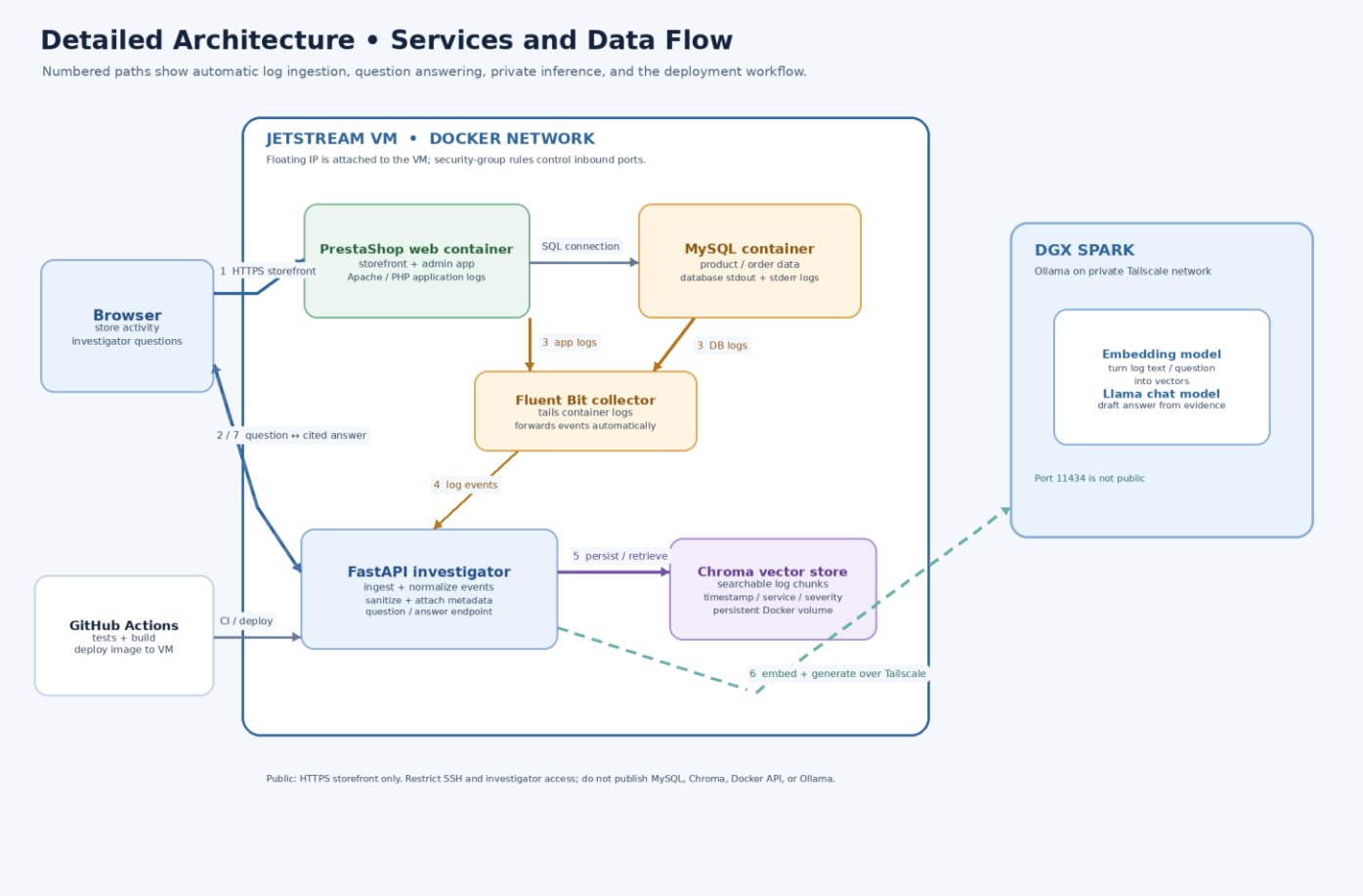
Scope

The minimum version covers one PrestaShop instance, one MySQL database, automatic log collection, a simple investigator question page, and one Jetstream deployment. Controlled test cases will include stopping or restarting the database, restarting the web container, and using a test-only invalid database setting. The system explains incidents but does not automatically change or restart services. Out of scope: real customer data or payment processing, production monitoring, automatic remediation, Kubernetes, high availability, and collecting logs from arbitrary public systems.

Detailed architecture and data flow (Initial)

The storefront and log-investigation service run in Docker on Jetstream. Fluent Bit forwards store and database logs to FastAPI, which indexes them in Chroma. FastAPI uses Ollama on the DGX Spark for embeddings and answer generation through Tailscale. GitHub Actions tests and builds the project before deployment

The diagram separates public store access from internal services and shows the two main paths: automatic ingestion of container logs and retrieval-augmented answers to user questions. The detailed implementation can start with one collector configuration and a small set of known incidents.



Main deployment choices

Jetstream: public storefront through a floating IP; allow only the required web traffic. Restrict SSH and the investigator interface. Jetstream private services: MySQL, Chroma, and the Docker API are not exposed publicly; persist the database and vector index with volumes. DGX Spark: Ollama serves the chat and embedding models only to the private tailnet; do not open Ollama to the internet.
Data: use synthetic products and test orders; redact secrets and sensitive fields before indexing logs.

Component responsibilities

Component	Runs on	Role

PrestaShop	Jetstream / Docker	Storefront and application logs from browsing and test orders.
MySQL	Jetstream / Docker	Store data; produces database logs and controlled outage cases.
Fluent Bit	Jetstream / Docker	Forwards container logs automatically to FastAPI.
FastAPI + Chroma	Jetstream / Docker	Normalizes events, retrieves relevant logs, and keeps the vector index.
Ollama + models	DGX Spark (Local)	Embeds log/question text and drafts an evidence-based answer.
GitHub Actions	GitHub	Runs tests and builds container images before deployment.

Cloud and DevOps workflow

Run PrestaShop and MySQL with Docker Compose locally; learn the basic store and inspect their container logs. Add the Fluent Bit configuration and verify that browsing, test orders, and controlled failures appear in the investigator service. Build the FastAPI question endpoint, Chroma persistence, retrieval logic, and Ollama calls; test locally against known log examples.

Use GitHub Actions to run unit tests and build images when code is pushed or a pull request is opened. Deploy containers to Jetstream; set security-group rules for web and SSH access; connect the VM to the DGX Spark with Tailscale. Verify restart behavior, volume persistence, Spark-unavailable handling, and the documented start / stop / reset process.

Proposed milestones

Week 1	Confirm Jetstream and Spark access; create repository and architecture.
Weeks 2–3	Run PrestaShop + MySQL in Compose; generate store activity and test incidents.
Week 4	Forward logs; build FastAPI ingestion and Chroma retrieval.
Week 5	Connect Ollama on DGX Spark through Tailscale; deploy to Jetstream.
Week 6	Add CI tests and evaluate known incident questions.
Week 7	Polish documentation, demo, and final report.

Evaluation plan

Use repeatable store actions and controlled failures to form a test set of about 20 questions. Score whether answers identify the known cause, cite the relevant log events, and avoid unsupported claims. Measure response time and test that logs continue to be collected when the DGX Spark is temporarily unreachable.

Risks and mitigations

PrestaShop setup takes longer than expected: begin with its documented Docker image and MySQL, then add the AI services after the store works. VM memory or disk is limited: keep the initial deployment small, monitor resource use, and keep model inference on the DGX Spark.

Logs contain secrets or personal data: use synthetic test orders, redact sensitive fields, and avoid real payment integrations. The model gives an unsupported diagnosis: include retrieved log evidence in the response and explicitly report uncertainty. DGX Spark or Tailscale is unavailable: keep the storefront and log collection running and show an AI-unavailable message. Public VM exposure: open only required storefront ports; keep database, Chroma, Docker API, and Ollama private.

Deliverables

GitHub repository with Docker Compose, collector configuration, FastAPI service, tests, and GitHub Actions workflow. PrestaShop demo on Jetstream connected privately to Ollama on DGX Spark. High-level and detailed architecture diagrams, README, and step-by-step tutorial. Evaluation results with sample incident questions, evidence-backed answers, response times, and limitations.
Final report and live class demonstration.

References

This project combines cloud infrastructure (Jetstream), containerization (Docker Compose), AI (RAG and local model inference), and DevOps (GitHub Actions and repeatable deployment). The first version stays focused on one e-commerce application and a few known incidents.

PrestaShop source and license: https://github.com/PrestaShop/PrestaShop
PrestaShop Docker image and usage: https://github.com/PrestaShop/docker
Course project ideas: https://cloudmesh-ai.github.io/cloudmesh-ai-lecture/lecture/assignments/ideas/

